Module 3, Comprendre les adresses IP et les sous-réseaux

Stéphane Robert
DevOps Engineer
Mis à jour le 29 juin 2026
Publié le 24 janvier 2026

Glossaire : ~~ACTIVÉ~~

21 min de lecture

Vous voyez passer des adresses comme `192.168.1.0/24` ou `10.0.0.0/8` et vous ne savez pas ce que signifie le `/24` ? Ce module vous explique comment lire une adresse IP, ce qu'est un masque de sous-réseau, et comment calculer combien de machines peuvent se connecter à un réseau. À la fin, vous saurez identifier une IP privée d'une publique et calculer une plage d'adresses.

TL;DR, L'essentiel en 30 secondes

- `/24` = masque 255.255.255.0 = 256 adresses (254 utilisables)
- **IP privées** : 10.x.x.x, 172.16-31.x.x, 192.168.x.x (ne sortent pas sur Internet)
- **CIDR** = notation `adresse/masque` (ex: 192.168.1.0/24)
- Plus le nombre après `/` est grand, plus le réseau est petit

Je sais que c'est bon si...

- ☐ Je sais que /24 = 254 machines utilisables
- ☐ Je reconnais une IP privée (192.168.x.x, 10.x.x.x)
- ☐ Je comprends que deux machines sur le même sous-réseau communiquent directement

Aide-mémoire rapide

Masque	Notation	Adresses	Machines utilisables
/8	255.0.0.0	16 millions	Très grands réseaux
/16	255.255.0.0	65 536	Entreprises
/24	255.255.255.0	256	Réseaux locaux
/32	255.255.255.255	1	Une seule machine

Prérequis

Pour suivre ce module, vous devez maîtriser :

- **Module 1** : ce qu'est une adresse IP, la différence LAN/Internet → **Revoir les bases absolues**
- **Module 2** : les commandes `ip addr`, `ip route` → **Revoir la boîte à outils**

> **RAPPEL EXPRESS**
> Une adresse IP est l'identifiant unique d'une machine sur un réseau. Elle permet aux machines de s'envoyer des données, comme une adresse postale permet de recevoir du courrier.

Ce que vous allez apprendre

- **Anatomie d'une adresse IPv4** : les 4 octets et leur signification
- **Le masque de sous-réseau** : ce que signifie `/24`, `/16`, `/8`
- **IP privées vs publiques** : pourquoi certaines adresses ne sortent pas sur Internet
- **Calculer une plage** : combien de machines dans un réseau `/24` ?
- **La notation CIDR** : lire et écrire `192.168.1.0/24`

Anatomie d'une adresse IPv4

Les 4 octets

Une adresse IPv4 est composée de **4 nombres** (appelés **octets**) séparés par des points :

```
192.168.1.45
```

Chaque octet peut prendre une valeur de **0 à 255** (soit 256 valeurs possibles). C'est logique : un octet = 8 bits, et 2^8 = 256.

Position	Octet 1	Octet 2	Octet 3	Octet 4
Valeur	192	168	1	45
Plage possible	0-255	0-255	0-255	0-255

Au total, une adresse IPv4 fait **32 bits** (4 × 8 = 32).

> **POURQUOI 4 MILLIARDS D'ADRESSES NE SUFFISENT PAS**
> Avec 32 bits, on peut créer 2^{32} = 4 294 967 296 adresses (environ 4,3 milliards). Ça semble énorme, mais avec des milliards d'appareils connectés aujourd'hui, ce n'est plus suffisant. C'est pour ça qu'on a inventé :
>
> - Les **IP privées** (réutilisables dans chaque réseau local)
> - **IPv6** (128 bits = un nombre astronomique d'adresses)

Partie réseau vs partie hôte

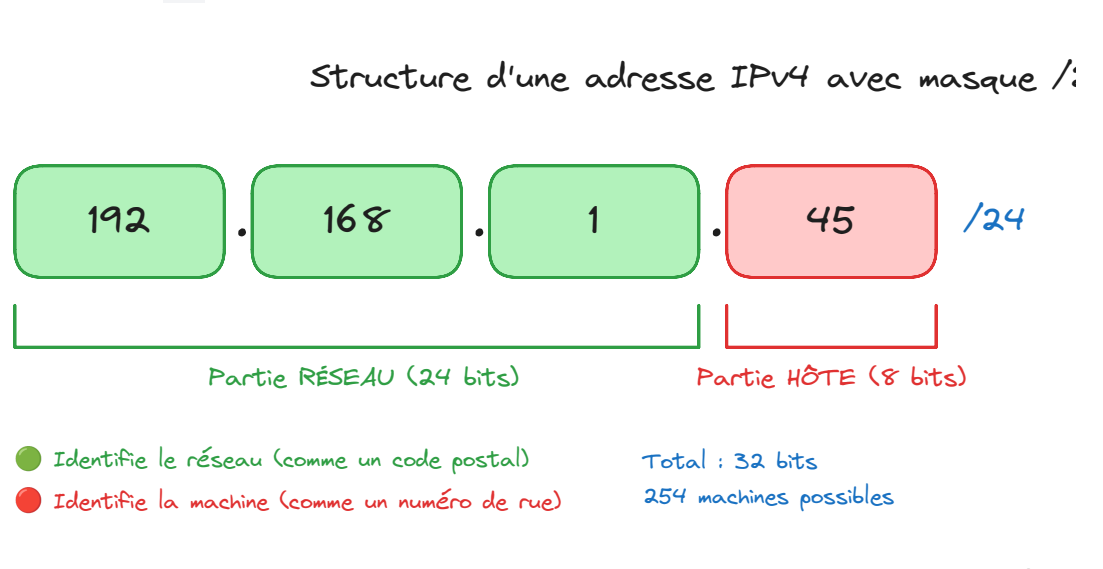
Voici le concept clé à comprendre : une adresse IP se divise en **deux parties** :

Partie	Rôle
Partie réseau	Identifie le réseau (le "quartier")
Partie hôte	Identifie la machine dans ce réseau

Prenons l'adresse `192.168.1.45` sur un réseau `/24` :

```
192.168.1  .  45
└────┬────┘   └┬┘
  Réseau      Hôte
```

- **Partie réseau** (`192.168.1`) : toutes les machines de ce réseau partagent ces 3 premiers octets
- **Partie hôte** (`45`) : identifiant unique de cette machine dans le réseau


Structure d'une adresse IPV4 avec masque /:
192
168
1
45
/24
Partie RÉSEAU (24 bits)
Partie HÔTE (8 bits)
🟢 Identifie le réseau (comme un code postal)
🔴 Identifie la machine (comme un numéro de rue)
Total : 32 bits
254 machines possibles

Conséquence : deux machines peuvent communiquer directement **seulement si elles partagent la même partie réseau**. Sinon, elles doivent passer par un routeur.

Le masque de sous-réseau

Le problème : où s'arrête le réseau ?

Quand vous voyez `192.168.1.45`, comment savez-vous où s'arrête la partie réseau et où commence la partie hôte ? C'est là qu'intervient le **masque de sous-réseau**.

Le masque indique **combien de bits sont réservés au réseau**. Plus le masque est grand, plus la partie réseau est importante, et moins il reste de place pour les machines.

La notation CIDR (le `/24`)

La **notation CIDR** (Classless Inter-Domain Routing) est la façon moderne d'écrire un masque :

```
192.168.1.0/24
```

Le `/24` signifie : **les 24 premiers bits identifient le réseau**.

Notation CIDR	Signification	Partie réseau	Partie hôte
`/24`	24 bits pour le réseau	3 octets (24 bits)	1 octet (8 bits)
`/16`	16 bits pour le réseau	2 octets (16 bits)	2 octets (16 bits)
`/8`	8 bits pour le réseau	1 octet (8 bits)	3 octets (24 bits)

La notation décimale (255.255.255.0)

Vous verrez aussi le masque écrit sous forme décimale. C'est la même information, présentée différemment :

CIDR	Masque décimal	Explication
`/24`	`255.255.255.0`	Les 3 premiers octets sont fixes (255 = tous les bits à 1)
`/16`	`255.255.0.0`	Les 2 premiers octets sont fixes
`/8`	`255.0.0.0`	Le premier octet est fixe

> **POURQUOI 255 ?**
> `255` en binaire = `11111111` (8 bits à 1). Un masque `255.255.255.0` signifie : "les 24 premiers bits (3 × 8) sont la partie réseau". Le `0` final signifie : "les 8 derniers bits sont libres pour les machines".

Voir votre masque

Sur votre machine **Linux**, le masque apparaît avec `ip addr` :

```
ip addr show | grep "inet "
```

Sortie typique :

```
inet 192.168.1.45/24 brd 192.168.1.255 scope global eth0
```

Le `/24` après votre IP est votre masque de sous-réseau.

Le cœur du calcul : l'opération AND

Pourquoi c'est important

Quand vous sortez des masques "faciles" (`/8`, `/16`, `/24`), le calcul devient moins intuitif. Pour vraiment comprendre, il faut connaître **une seule règle** : l'opération **AND binaire**.

La règle fondamentale

```
Adresse réseau = IP AND masque
```

C'est cette opération que fait votre système pour déterminer si deux machines sont sur le même réseau.

Exemple avec `/24` (le plus simple) :

```
IP :     192.168.1.45    → 11000000.10101000.00000001.00101101
Masque : 255.255.255.0   → 11111111.11111111.11111111.00000000
                           ───────────────────────────────────
Réseau : 192.168.1.0     → 11000000.10101000.00000001.00000000
```

Partout où le masque a un `1`, on garde le bit de l'IP. Partout où il a un `0`, le résultat est `0`.

Le broadcast : l'inverse

```
Broadcast = réseau OR (NOT masque)
```

On met tous les bits "hôte" à 1 :

```
Réseau :     192.168.1.0     → 11000000.10101000.00000001.00000000
NOT masque : 0.0.0.255       → 00000000.00000000.00000000.11111111
                               ───────────────────────────────────
Broadcast :  192.168.1.255   → 11000000.10101000.00000001.11111111
```

> **MÉTHODE RAPIDE : MÊME RÉSEAU ?**
> Deux machines sont sur le même réseau si :
>
> ```
> (IP1 AND masque) == (IP2 AND masque)
> ```
>
> En pratique, utilisez :
>
> ```
> # Linux vous dit directement par où passer
> ip route get 10.45.128.100
> ```
>
> Si la sortie montre `via <gateway>`, les machines sont sur des réseaux différents. Si elle montre `dev eth0` sans `via`, elles sont sur le même réseau.

Calculer une plage d'adresses

Combien de machines dans un réseau ?

C'est une question fréquente : "Si j'ai un réseau `/24`, combien de machines puis-je y connecter ?"

La formule est simple :

```
Nombre d'adresses = 2^(32 - masque)
Machines utilisables = Nombre d'adresses - 2
```

On soustrait 2 car deux adresses sont **toujours réservées** (sauf pour `/31`) :

- **Adresse réseau** (première) : identifie le réseau lui-même
- **Adresse de broadcast** (dernière) : envoie à toutes les machines du réseau

Masque	Adresses totales	Machines utilisables	Cas d'usage typique
`/32`	1	1	Route host (IP unique), ACL, tunnels
`/31`	2	2	Liaison point-à-point moderne (RFC 3021)
`/30`	4	2	Liaison point-à-point classique
`/28`	16	14	Petit réseau (équipe)
`/26`	64	62	Sous-réseau département
`/24`	256	254	Réseau local standard
`/16`	65 536	65 534	Grande entreprise, VPC cloud
`/8`	16 777 216	16 777 214	Très grand réseau

> **CAS SPÉCIAUX /31 ET /32**
> `/31` : Normalisé par **RFC 3021**, ce masque permet **2 adresses utilisables** (pas de broadcast ni d'adresse réseau "sacrifiée"). Très courant sur les liens point-à-point entre routeurs.
> `/32` : Une seule adresse. Utilisé pour les **routes host** (routage vers une IP précise), les règles de firewall, ou les interfaces de loopback.

Exemple concret : réseau 192.168.1.0/24

Décomposons le réseau `192.168.1.0/24` :

Élément	Adresse	Rôle
Adresse réseau	`192.168.1.0`	Identifie le réseau (non assignable)
Première IP utilisable	`192.168.1.1`	Souvent la passerelle (box/routeur)
Dernière IP utilisable	`192.168.1.254`	Dernière machine possible
Adresse de broadcast	`192.168.1.255`	Envoie à tout le monde (non assignable)
Plage utilisable	`192.168.1.1` → `192.168.1.254`	254 machines

> **ASTUCE MÉMO POUR /24**
> Pour un réseau `/24`, c'est facile :
>
> - L'adresse réseau finit par `.0`
> - Le broadcast finit par `.255`
> - Vous avez **254 machines** possibles

Calculer sans ipcalc : la méthode des blocs

Quand le masque n'est **pas un multiple de 8** (`/26`, `/20`, `/27` ...), le dernier octet concerné ne tombe plus sur 0 ou 255. Il faut raisonner en **blocs**.

Le concept de "block size"

La **taille du bloc** (block size) = 2^(bits restants dans l'octet concerné).

Masque	Bits hôte dans l'octet	Block size	Limites du dernier octet
`/26`	6 bits	64	0, 64, 128, 192
`/27`	5 bits	32	0, 32, 64, 96, 128, 160, 192, 224
`/28`	4 bits	16	0, 16, 32, 48, 64, 80, ...
`/29`	3 bits	8	0, 8, 16, 24, 32, 40, ...
`/30`	2 bits	4	0, 4, 8, 12, 16, ...

Exemple : 192.168.1.100/26

1. **Identifier le block size**

   `/26` = 26 bits réseau, donc 6 bits hôte (32 - 26 = 6). Block size = 2^6 = **64**.

2. **Trouver les limites de bloc**

   Blocs : 0, 64, 128, 192. L'IP `100` est entre **64** et **128**.

3. **Calculer réseau et broadcast**

   - Adresse réseau : `192.168.1.64`
   - Broadcast : `192.168.1.127` (64 + 63)
   - Plage : `192.168.1.65` → `192.168.1.126` (62 machines)

Exemple avancé : 10.45.128.67/20

Le `/20` touche le **troisième octet** (pas le quatrième).

1. **Identifier l'octet concerné**

   `/20` = 20 bits réseau = 2 octets complets (16 bits) + 4 bits dans le 3e octet. Les bits hôte commencent au 3e octet.

2. **Calculer le block size**

   Bits dans le 3e octet pour les hôtes : 8 - 4 = 4 bits → block size = 2^4 = **16**. Mais attention : les 8 bits du 4e octet sont aussi pour les hôtes, donc le réseau avance par **blocs de 16 sur le 3e octet**.

3. **Trouver les limites**

   Blocs du 3e octet : 0, 16, 32, 48, 64, 80, 96, 112, **128**, 144, ... `128` est pile sur une limite → c'est l'adresse réseau.

4. **Résultat**

   - Réseau : `10.45.128.0/20`
   - Broadcast : `10.45.143.255` (128 + 15 = 143 pour le 3e octet)
   - Plage : `10.45.128.1` → `10.45.143.254` (4094 machines)

> **VÉRIFICATION AVEC IPCALC**
> Après votre calcul mental, validez toujours avec `ipcalc` :
>
> ```
> ipcalc 10.45.128.67/20
> ```
>
> ```
> Network:   10.45.128.0/20
> Broadcast: 10.45.143.255
> HostMin:   10.45.128.1
> HostMax:   10.45.143.254
> Hosts/Net: 4094
> ```

TP : calculer la plage de votre réseau

1. **Affichez votre configuration réseau**

   ```
   ip addr show | grep "inet " | grep -v "127.0.0.1"
   ```

   Notez votre IP et votre masque (exemple : `192.168.1.45/24`).

2. **Identifiez l'adresse réseau**

   Avec un masque `/24`, gardez les 3 premiers octets et mettez le dernier à 0.

   → `192.168.1.45` devient `192.168.1.0` (adresse réseau)

3. **Calculez le broadcast**

   Avec un masque `/24`, gardez les 3 premiers octets et mettez le dernier à 255.

   → Broadcast = `192.168.1.255`

4. **Vérifiez avec `ipcalc`** (installez-le si nécessaire)

   ```
   # Installation sur Debian/Ubuntu
   sudo apt install ipcalc

   # Calcul
   ipcalc 192.168.1.45/24
   ```

   Sortie :

   ```
   Address:   192.168.1.45
   Network:   192.168.1.0/24
   Netmask:   255.255.255.0 = 24
   Broadcast: 192.168.1.255

   HostMin:   192.168.1.1
   HostMax:   192.168.1.254
   Hosts/Net: 254
   ```

Adresses spéciales à connaître

Les trois plages **privées** que vous avez croisées plus haut sous la forme `10.x.x.x` portent un **nom** et des **bornes exactes** : ce sont les blocs de la **RFC 1918**, et les écrire en **notation CIDR** évite de les confondre avec les autres plages réservées.

Plage privée (RFC 1918)	Notation CIDR	Nombre d'adresses
`10.0.0.0` à `10.255.255.255`	`10.0.0.0/8`	~16,7 millions
`172.16.0.0` à `172.31.255.255`	`172.16.0.0/12`	~1 million
`192.168.0.0` à `192.168.255.255`	`192.168.0.0/16`	~65 000

Retenez surtout le `/12` de la plage médiane, seule irrégularité de la série : elle ne couvre **pas** tout `172.x.x.x`, mais seulement de `172.16` à `172.31`. Une adresse en `172.15.0.1` ou `172.32.0.1` est donc **publique**, et la poser sur un réseau local revient à s'approprier des adresses appartenant à quelqu'un d'autre.

D'autres adresses ont un rôle particulier, sans être privées pour autant :

Adresse	Nom	Utilité
`127.0.0.1`	**Localhost**	Votre propre machine (boucle locale)
`0.0.0.0`	**Toutes les interfaces**	"Écouter sur toutes les IP"
`255.255.255.255`	**Broadcast global**	Envoyer à tout le monde (rarement utilisé)
`169.254.x.x`	**Link-local (RFC 3927)**	IP auto-attribuée quand pas de **DHCP**
`100.64.0.0/10`	**CGNAT (RFC 6598)**	Utilisé par les FAI pour le NAT partagé

> **SI VOUS VOYEZ 169.254.X.X**
> Une adresse `169.254.x.x` (plage link-local **RFC 3927**) signifie que votre machine n'a **pas réussi à obtenir une IP** via DHCP. Vérifiez :
>
> - Votre câble réseau
> - Que le Wi-Fi est connecté
> - Que le serveur DHCP (votre box) fonctionne

> **CGNAT : ATTENTION AUX CONFUSIONS**
> Si vous voyez une IP en `100.64.x.x` à `100.127.x.x`, ce n'est **pas** une IP privée, mais une adresse **CGNAT**. Votre FAI (ou opérateur 4G/5G) l'utilise pour faire du NAT **avant** de vous attribuer une IP publique.
> Conséquences :
>
> - Vous ne pouvez **pas héberger de serveur** accessible de l'extérieur
> - Plusieurs clients partagent la même IP publique
> - Courant chez les opérateurs mobiles et certaines box fibre
>
> ```
> # Vérifier si vous êtes derrière du CGNAT
> ip addr | grep "100\.\(6[4-9]\|[7-9][0-9]\|1[01][0-9]\|12[0-7]\)\."
> ```

Cas pratique : analyser votre réseau d'entreprise

Imaginons que vous arrivez sur un serveur en production et que vous voyez :

```
$ ip addr show eth0
inet 10.45.128.67/20 brd 10.45.143.255 scope global eth0
```

Que pouvez-vous en déduire ?

1. **Type d'adresse**

   `10.x.x.x` → C'est une **IP privée** (plage entreprise/cloud).

2. **Taille du réseau**

   `/20` = 2^(32-20) = 2^12 = **4096 adresses** (4094 machines utilisables).

3. **Adresse réseau**

   C'est plus complexe qu'un `/24`. Utilisons `ipcalc` :

   ```
   ipcalc 10.45.128.67/20
   ```

   ```
   Network:   10.45.128.0/20
   Broadcast: 10.45.143.255
   HostMin:   10.45.128.1
   HostMax:   10.45.143.254
   Hosts/Net: 4094
   ```

4. **Interprétation**

   Ce serveur fait partie d'un réseau de 4094 machines (probablement un sous-réseau cloud ou datacenter).

TP : explorer les sous-réseaux courants

Utilisez `ipcalc` pour explorer différentes configurations :

1. **Réseau domestique classique**

   ```
   ipcalc 192.168.1.0/24
   ```

   → 254 machines, parfait pour une maison ou une petite entreprise.

2. **Réseau cloud AWS/Azure typique**

   ```
   ipcalc 10.0.0.0/16
   ```

   → 65 534 machines, un VPC entier.

3. **Sous-réseau Kubernetes**

   ```
   ipcalc 10.244.0.0/16
   ```

   → Plage typique pour les **pods** Kubernetes.

4. **Liaison point-à-point**

   ```
   ipcalc 192.168.100.0/30
   ```

   → Seulement 2 machines : un routeur et un serveur.

> **EXERCICE BONUS**
> Trouvez le sous-réseau de GitHub :
>
> ```
> dig github.com +short
> ipcalc <IP_obtenue>/24
> ```
>
> Vous verrez que GitHub utilise des IP publiques, pas des plages privées comme votre réseau local.

Erreurs courantes et dépannage

"Network unreachable" (réseau inaccessible)

Symptôme : vous ne pouvez pas joindre une machine, même avec ping.

Causes possibles :

Vérification	Commande	Solution
Pas dans le même sous-réseau	`ip addr`	Vérifier que les machines partagent le même préfixe réseau
Pas de route vers ce réseau	`ip route`	Ajouter une route ou vérifier la passerelle
Masque incorrect	`ip addr`	Corriger le masque (ex: `/24` au lieu de `/32`)

"No route to host" (pas de route)

Symptôme : ping vers une IP échoue avec ce message.

Diagnostic :

```
# Vérifier votre table de routage
ip route

# Vérifier si vous avez une passerelle par défaut
ip route | grep default
```

Si vous n'avez pas de `default via`, votre machine ne sait pas comment atteindre les réseaux extérieurs.

"Destination Host Unreachable" (problème L2/ARP)

Symptôme : ping échoue avec ce message, même si la destination est sur le même réseau.

Cause : la machine source ne peut pas résoudre l'adresse MAC de la destination (problème au niveau 2).

Diagnostic :

```
# Vérifier le cache ARP
ip neigh
# ou
arp -n

# Chercher les entrées FAILED ou INCOMPLETE
ip neigh | grep -E "FAILED|INCOMPLETE"
```

Causes possibles :

- La machine destination est éteinte ou déconnectée
- VLAN différent (même si même sous-réseau IP)
- Problème de câble ou de switch
- Adresse IP en conflit

Ping OK mais TCP KO (problème firewall)

Symptôme : `ping 192.168.1.100` fonctionne, mais `curl http://192.168.1.100` timeout.

Cause : un **firewall** bloque le port TCP, mais pas **ICMP** (ping).

Diagnostic :

```
# Sur la machine destination, vérifier que le service écoute
ss -tlnp | grep :80

# Vérifier les règles firewall (iptables)
sudo iptables -L -n | grep -i drop

# Vérifier les règles firewall (nftables)
sudo nft list ruleset | grep -i drop

# Vérifier firewalld (si utilisé)
sudo firewall-cmd --list-all
```

> **ORDRE DE DIAGNOSTIC RÉSEAU**
>
> 1. **Ping local** : `ping 127.0.0.1` (problème système si KO)
> 2. **Ping passerelle** : `ping <gateway>` (problème réseau local si KO)
> 3. **Ping Internet** : `ping 8.8.8.8` (problème routage si KO)
> 4. **Ping DNS** : `ping google.com` (problème DNS si KO)

IP en 169.254.x.x

Symptôme : votre machine a une IP `169.254.x.x`.

Cause : le client DHCP n'a pas réussi à obtenir une adresse.

Solutions :

```
# Renouveler le bail DHCP
sudo dhclient -r eth0  # libère l'IP
sudo dhclient eth0     # demande une nouvelle IP

# Ou redémarrer le service réseau
sudo systemctl restart NetworkManager
```

À retenir

1. **Une adresse IPv4** = 4 octets (32 bits), valeurs de 0 à 255
2. **Le masque** (ex: `/24`) indique combien de bits sont réservés au réseau
3. **Adresse réseau** = IP AND masque (le vrai calcul derrière tout)
4. `/24` = 256 adresses, **254 utilisables** (le plus courant)
5. **IP privées** : `10.x.x.x`, `172.16-31.x.x`, `192.168.x.x`, non routées sur Internet
6. **CGNAT** (`100.64.x.x`) : ce n'est **pas** du RFC 1918, c'est votre FAI
7. **Hors /8-/16-/24** : raisonnez par **blocs** (block size = 2^bits_hôte)
8. `ipcalc` est votre meilleur ami pour vérifier vos calculs
9. `ip route get <IP>` : la commande diagnostic ultime
10. **169.254.x.x** = problème DHCP, pas d'IP attribuée

Testez vos connaissances

Contrôle de connaissances

Validez vos connaissances avec ce quiz interactif

10 questions · **10** min. · **80%** requis

INFORMATIONS

- Le chronomètre démarre au clic sur *Démarrer*
- Questions à choix multiples, vrai/faux et réponses courtes
- Vous pouvez naviguer entre les questions
- Les résultats détaillés sont affichés à la fin

Démarrer le quiz

FAQ : questions fréquentes sur les sous-réseaux

Qu'est-ce qu'un masque de sous-réseau ?
Le masque de sous-réseau sépare une adresse IP en deux parties : la…

Combien d'hôtes peut-on adresser dans un /24 ?
Un /24 contient 256 adresses (2^8), mais seulement 254 sont utilisables…

Quelle est la différence entre adresse réseau et adresse de broadcast ?
L'adresse réseau (bits hôte à 0) identifie le segment, elle ne désigne…

Comment découper un réseau en sous-réseaux (VLSM) ?
Le VLSM (Variable Length Subnet Mask) découpe un réseau en sous-…

Que signifie la notation /24 (CIDR) ?
La notation CIDR /24 indique le nombre de bits réservés à la partie rése…

Comment calculer l'adresse réseau et le broadcast ?
L'adresse réseau s'obtient en mettant tous les bits de la partie hôte à 0, l…

Les classes A, B et C existent-elles encore ?
Non, l'adressage par classes (classful) est obsolète depuis CIDR en 199…

Prochaines étapes

Module 4 : Le routage →
Comment les paquets trouvent leur chemin entre différents réseaux

Module 5 : TCP vs UDP →
Les protocoles de transport : fiabilité, ports, netcat

Retour au parcours →
Vue d'ensemble de la formation réseau

Un problème sur cette page ?

Erreur dans la page · Contenu obsolète · Lien cassé · Amélioration

Choisissez le type : l'issue s'ouvre déjà remplie avec le titre et l'adresse de cette page.

© 2026 DevSecOps • Mentions légales